StockFlow

Inventory Management System

Backend Engineering Intern — Case Study Solution

Language / Framework	Node.js + Express.js
Database	PostgreSQL (Sequelize ORM)
Sections	Part 1: Code Review | Part 2: DB Design | Part 3: API

Part 1 — Code Review & Debugging

1.1 Original Code (Python/Flask)

The intern submitted the following endpoint to create a product:

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json
    # Create new product
    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )
    db.session.add(product)
    db.session.commit()      # Commit #1
    # Update inventory count
    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )
    db.session.add(inventory)
```

```
    db.session.commit()      # Commit #2
    return {"message": "Product created", "product_id": product.id}
```

1.2 Issues Identified

Issue 1 — No Input Validation

Impact: If name, sku, price, or warehouse_id are missing, Python raises a KeyError → unhandled 500 error exposed to the client. Malformed price values can corrupt the database.

Issue 2 — Two Separate Transactions (Non-Atomic)

Impact: If the server crashes after the first db.session.commit(), a Product row exists in the DB with no matching Inventory row. The product is 'ghost' stock — visible but untraceable. This is a critical data-consistency bug.

Issue 3 — warehouse_id Stored on the Product Model

Impact: The spec states products can exist in multiple warehouses. Storing warehouse_id on the product table enforces a 1-to-1 relationship and breaks multi-warehouse support.

Issue 4 — No SKU Uniqueness Check

Impact: Two concurrent requests with the same SKU could both pass the Product creation step before either commits, resulting in duplicate SKUs — violating the business rule and causing query ambiguity.

Issue 5 — Price Stored Without Decimal Precision Guarantee

Impact: Passing an unvalidated float for price can result in floating-point rounding errors (e.g. $19.99 stored as 19.990000000001). NUMERIC(12,2) in the DB schema avoids this, but the API must also validate.

Issue 6 — initial_quantity Not Optional / No Default

Impact: If initial_quantity is missing, Python raises KeyError. Logically, a newly listed product with no stock is valid — the field should default to 0.

Issue 7 — No Error Handling / No HTTP Status Codes

Impact: All success and failure paths return an implicit 200. REST clients cannot distinguish created (201), bad request (400), conflict (409), or server error (500).

1.3 Fixed Implementation (Node.js / Express)

```
// ─── models (abbreviated) ──────────────────────────────────
// Product, Inventory, Company defined with Sequelize / Prisma

const express = require('express');
const { body, param, validationResult } = require('express-validator');
const { sequelize, Product, Inventory } = require('./models');
const router = express.Router();

// POST /api/products
router.post('/',
  // 1. Input validation middleware
  [
    body('name').notEmpty().withMessage('name is required'),
    body('sku').notEmpty().withMessage('sku is required'),
    body('price').isDecimal({ decimal_digits: '0,2' }).withMessage('price must be
a valid decimal'),
    body('warehouse_id').isInt().withMessage('warehouse_id must be an integer'),
    body('initial_quantity').optional().isInt({ min: 0 }).withMessage('quantity
must be >= 0'),
  ],
  async (req, res) => {
    // 2. Validation errors → 400
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }

    const { name, sku, price, warehouse_id, initial_quantity = 0 } = req.body;

    // 3. Wrap in a single transaction — atomic: both rows saved or neither
    const t = await sequelize.transaction();
    try {
      // 4. Enforce SKU uniqueness at DB level (unique constraint) + app level
      const existing = await Product.findOne({ where: { sku }, transaction: t });
      if (existing) {
        await t.rollback();
        return res.status(409).json({ error: `SKU '${sku}' already exists` });
      }

      // 5. Create product — warehouse_id removed (product lives in many
warehouses)
      const product = await Product.create({ name, sku, price }, { transaction: t
});

      // 6. Create initial inventory record for this warehouse
      await Inventory.create(
        { product_id: product.id, warehouse_id, quantity: initial_quantity },
        { transaction: t }
      );
```

```
      await t.commit();
      return res.status(201).json({ message: 'Product created', product_id:
product.id });

    } catch (err) {
      await t.rollback();
      console.error('[create_product]', err);
      return res.status(500).json({ error: 'Internal server error' });
    }
  }
);

module.exports = router;
```

1.4 Summary of Fixes

Fix	Technique
Input validation	express-validator on all fields
Atomicity	Single transaction wrapping both INSERT statements
SKU uniqueness	findOne check inside transaction + DB UNIQUE constraint
Decimal precision	isDecimal validator + NUMERIC(12,2) column
Optional quantity	Defaults to 0 when not provided
HTTP status codes	201 Created, 400 Bad Request, 409 Conflict, 500 Error
warehouse_id on product	Removed — relationship lives in inventory table only

Part 2 — Database Design

2.1 Entity-Relationship Overview

The schema below satisfies all given requirements plus reasonable assumptions for a production B2B SaaS product.

Table	Key Columns	Relationships
companies	`id, name`	1 → N warehouses, suppliers, products
warehouses	`id, company_id, name, location`	N → 1 companies
suppliers	`id, company_id, name, contact_email`	N → 1 companies; 1 → N products

products	`id, company_id, supplier_id, sku (UNIQUE), price, low_stock_threshold`	N → 1 companies; N-to-N warehouses via inventory
product_bundles	`bundle_id, component_id, quantity`	Self-join on products
inventory	`id, product_id, warehouse_id, quantity — UNIQUE(product, warehouse)`	Junction table: products ↔ warehouses
inventory_history	`id, inventory_id, delta, reason, created_at`	Audit log for every inventory change
sales	`id, product_id, warehouse_id, quantity_sold, sold_at`	Used to compute recent activity & daily sell-rate

2.2 Full SQL DDL

```
-- ─── COMPANIES ──────────────────────────────────────────────
CREATE TABLE companies (
  id            SERIAL PRIMARY KEY,
  name          VARCHAR(255) NOT NULL,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- ─── WAREHOUSES ─────────────────────────────────────────────
CREATE TABLE warehouses (
  id            SERIAL PRIMARY KEY,
  company_id    INT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  name          VARCHAR(255) NOT NULL,
  location      TEXT,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_warehouses_company ON warehouses(company_id);

-- ─── SUPPLIERS ──────────────────────────────────────────────
CREATE TABLE suppliers (
  id            SERIAL PRIMARY KEY,
  company_id    INT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  name          VARCHAR(255) NOT NULL,
  contact_email VARCHAR(320),
  contact_phone VARCHAR(50),
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_suppliers_company ON suppliers(company_id);

-- ─── PRODUCTS ───────────────────────────────────────────────
CREATE TABLE products (
  id            SERIAL PRIMARY KEY,
  company_id    INT NOT NULL REFERENCES companies(id),
  supplier_id   INT REFERENCES suppliers(id) ON DELETE SET NULL,
  name          VARCHAR(255) NOT NULL,
  sku           VARCHAR(100) NOT NULL UNIQUE,   -- global uniqueness
  price         NUMERIC(12,2) NOT NULL CHECK (price >= 0),
```

```
  product_type  VARCHAR(50) NOT NULL DEFAULT 'standard',
  low_stock_threshold INT NOT NULL DEFAULT 10,
  is_active     BOOLEAN NOT NULL DEFAULT TRUE,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_products_company ON products(company_id);
CREATE INDEX idx_products_sku     ON products(sku);

-- ─── PRODUCT BUNDLES (self-referential) ──────────────────
CREATE TABLE product_bundles (
  bundle_id     INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  component_id  INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  quantity      INT NOT NULL DEFAULT 1 CHECK (quantity > 0),
  PRIMARY KEY (bundle_id, component_id)
);

-- ─── INVENTORY ───────────────────────────────────────────
CREATE TABLE inventory (
  id            SERIAL PRIMARY KEY,
  product_id    INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  warehouse_id  INT NOT NULL REFERENCES warehouses(id) ON DELETE CASCADE,
  quantity      INT NOT NULL DEFAULT 0 CHECK (quantity >= 0),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (product_id, warehouse_id)
);
CREATE INDEX idx_inventory_product   ON inventory(product_id);
CREATE INDEX idx_inventory_warehouse ON inventory(warehouse_id);

-- ─── INVENTORY HISTORY (audit log) ───────────────────────
CREATE TABLE inventory_history (
  id            SERIAL PRIMARY KEY,
  inventory_id  INT NOT NULL REFERENCES inventory(id),
  changed_by    INT REFERENCES users(id),
  delta         INT NOT NULL,   -- +/- change in quantity
  reason        VARCHAR(100),   -- 'sale', 'purchase', 'adjustment', 'transfer'
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_inv_history_inventory ON inventory_history(inventory_id);
CREATE INDEX idx_inv_history_created   ON inventory_history(created_at);

-- ─── SALES (for recent activity detection) ───────────────
CREATE TABLE sales (
  id            SERIAL PRIMARY KEY,
  product_id    INT NOT NULL REFERENCES products(id),
  warehouse_id  INT NOT NULL REFERENCES warehouses(id),
  quantity_sold INT NOT NULL CHECK (quantity_sold > 0),
  sold_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_sales_product_date ON sales(product_id, sold_at);
```

2.3 Design Decisions & Justifications

Atomicity & Constraints

- UNIQUE constraint on products.sku prevents duplicate SKUs at the database level even under concurrent writes.

- UNIQUE(product_id, warehouse_id) on inventory ensures only one stock record per product-warehouse pair.
- CHECK (quantity >= 0) prevents negative stock at the database level.

Multi-Warehouse Support

- warehouse_id is NOT on the products table. The inventory table is the junction table implementing the many-to-many relationship between products and warehouses.

Product Bundles

- product_bundles is a self-referential association table. A bundle product is simply a product whose id appears as bundle_id in this table.
- This approach avoids a separate 'bundle' entity and re-uses all product logic (pricing, inventory, SKU).

Audit / History

- inventory_history stores every delta (positive for restocks, negative for sales/adjustments) along with a reason code and actor (changed_by).
- The sales table is kept separate to enable fast aggregations for sell-rate calculations without scanning the full history log.

Indexes

- Composite index on sales(product_id, sold_at) is critical for the low-stock alert query which filters by date range per product.
- All foreign keys are indexed to avoid full-table scans on JOIN operations.

low_stock_threshold per Product

- Stored directly on the products table (default 10). This allows different thresholds per product type without a separate config table, satisfying the Part 3 business rule.

2.4 Questions for the Product Team (Identified Gaps)

1. User/Auth model — which table owns user accounts? Is authentication per-company or global?
2. Bundle inventory — does a bundle deduct stock from individual components on sale, or does it carry its own inventory record?
3. Supplier–product relationship — can one product have multiple suppliers? (currently 1 supplier per product)
4. Soft deletes — should deleted products/warehouses be archived rather than hard-deleted?
5. Multi-currency — does price need a currency_code column, or is the platform single-currency?
6. 'Recent sales activity' window — is 30 days the correct definition of recent, or is it configurable?

7. Reorder quantity — should the schema store a reorder_quantity per product for automatic PO generation?
8. Transfer between warehouses — is stock movement between a company's own warehouses a first-class operation?

Part 3 — API Implementation

3.1 Endpoint Specification

Endpoint: GET /api/companies/:company_id/alerts/low-stock

3.2 Assumptions Made

- 'Recent sales activity' is defined as at least one sale within the last 30 days.
- days_until_stockout = floor(current_stock / avg_daily_sales_last_30_days). Returns null when avg_daily_sales = 0.
- low_stock_threshold is stored per-product row (Part 2 schema). This naturally supports different thresholds per product type.
- Authentication middleware has already validated the caller has access to company_id.
- Products marked is_active = FALSE are excluded from alerts.

3.3 Implementation (Node.js / Express + Sequelize)

```
// GET /api/companies/:company_id/alerts/low-stock
// Returns low-stock alerts for all warehouses of a company.

const express  = require('express');
const { param } = require('express-validator');
const { sequelize } = require('./models');
const router   = express.Router();

// ── ASSUMPTIONS ──────────────────────────────────────────────
// 1. 'Recent sales activity' = at least 1 sale in last 30 days.
// 2. 'days_until_stockout' = current_stock / avg_daily_sales (30-day window).
//    Returns null when avg_daily_sales = 0 (no recent activity).
// 3. Each product has a per-row low_stock_threshold (set by product type).
// 4. Auth middleware already validates company access (req.user.company_id).
// ─────────────────────────────────────────────────────────────

const RECENT_DAYS = 30;

router.get('/:company_id/alerts/low-stock',
  [param('company_id').isInt().withMessage('company_id must be an integer')],
  async (req, res) => {
```

```
  const { company_id } = req.params;

  try {
    // ── Single optimised SQL query ──────────────────────────
    // CTE 1: total sales per product/warehouse in last 30 days
    // CTE 2: avg daily sales rate
    // Main : join inventory + product + warehouse + supplier
    //        filter to rows below threshold with recent activity
    const [alerts] = await sequelize.query(`
      WITH recent_sales AS (
        SELECT
          s.product_id,
          s.warehouse_id,
          SUM(s.quantity_sold)                        AS total_sold,
          SUM(s.quantity_sold)::FLOAT / :recent_days  AS avg_daily_sales
        FROM sales s
        JOIN warehouses w ON w.id = s.warehouse_id
        WHERE w.company_id = :company_id
          AND s.sold_at >= NOW() - INTERVAL ':recent_days days'
        GROUP BY s.product_id, s.warehouse_id
      )
      SELECT
        p.id              AS product_id,
        p.name            AS product_name,
        p.sku,
        w.id              AS warehouse_id,
        w.name            AS warehouse_name,
        inv.quantity      AS current_stock,
        p.low_stock_threshold AS threshold,
        CASE
          WHEN rs.avg_daily_sales > 0
          THEN FLOOR(inv.quantity / rs.avg_daily_sales)
          ELSE NULL
        END               AS days_until_stockout,
        sup.id            AS supplier_id,
        sup.name          AS supplier_name,
        sup.contact_email AS supplier_email
      FROM inventory inv
      JOIN products   p   ON p.id  = inv.product_id
      JOIN warehouses w   ON w.id  = inv.warehouse_id
      LEFT JOIN suppliers sup ON sup.id = p.supplier_id
      JOIN recent_sales rs
        ON rs.product_id   = inv.product_id
       AND rs.warehouse_id = inv.warehouse_id
      WHERE w.company_id        = :company_id
        AND p.is_active         = TRUE
        AND inv.quantity        < p.low_stock_threshold
      ORDER BY days_until_stockout ASC NULLS LAST
    `, {
      replacements: { company_id, recent_days: RECENT_DAYS },
      type: sequelize.QueryTypes.SELECT
    });

    // ── Shape the response ──────────────────────────────────
    const shaped = alerts.map(row => ({
      product_id:         row.product_id,
      product_name:       row.product_name,
      sku:                row.sku,
      warehouse_id:       row.warehouse_id,
      warehouse_name:     row.warehouse_name,
      current_stock:      row.current_stock,
```

```
        threshold:          row.threshold,
        days_until_stockout: row.days_until_stockout !== null
                              ? Number(row.days_until_stockout)
                              : null,
        supplier: row.supplier_id ? {
          id:            row.supplier_id,
          name:          row.supplier_name,
          contact_email: row.supplier_email,
        } : null,
      }));

      return res.status(200).json({
        alerts:       shaped,
        total_alerts: shaped.length,
      });

    } catch (err) {
      console.error('[low-stock-alerts]', err);
      return res.status(500).json({ error: 'Internal server error' });
    }
  }
);

module.exports = router;
```

3.4 Edge Cases Handled

Edge Case	Handling
No alerts found	Returns { alerts: [], total_alerts: 0 } — not a 404
Product has no supplier	supplier field is null in response (LEFT JOIN)
Zero daily sales rate	days_until_stockout returns null (avoids divide-by-zero)
Invalid company_id	express-validator returns 400 before hitting the DB
Non-existent company_id	Empty array returned — no DB error
DB error	try/catch logs error server-side; client receives 500 JSON
Inactive products	Filtered out via WHERE p.is_active = TRUE
Multiple warehouses	Each product/warehouse pair is its own alert row

3.5 Performance Notes

- A single SQL query with CTEs avoids N+1 query problems — no per-product lookups in application code.
- The composite index on sales(product_id, sold_at) makes the 30-day aggregation efficient even with millions of sales rows.
- Result can be cached (Redis, 60-second TTL) for high-traffic companies since alerts do not need real-time precision.
- Pagination (LIMIT / OFFSET) should be added if total_alerts can grow large (> 500 rows).